Deliverable 5 — Deployment and Final Report

Healthcare Web Application

Course Code — CSE 616
Course Name — Web Engineering Lab

Group Number: 14

Group Members:
Israt Zannat — 22701027
Sajid Shahriar Sanam — 22701031
Shajeda Isnat — 22701060

Course Instructor:
Professor Rokan Uddin Faruqui

Department of Computer Science and Engineering,
University of Chittagong

Date Prepared: November 30, 2025

Contents

1 Introduction **5**

2 System Overview **5**

3 Deployment Architecture **6**
3.1 Deployment Overview . . . 6
3.2 System Interaction Flow . . . 7
3.3 Deployment Platforms Used . . . 8
3.4 Environment Configuration . . . 8
3.5 Live Deployment Links . . . 9
3.6 Deployment Architecture Diagram . . . 9

4 Docker Containerization **10**
4.1 Concept of Containerization . . . 10
4.2 Docker in This Project . . . 11
4.3 Sample Dockerfile (Frontend) . . . 11
4.4 Future Containerization Plan . . . 11
4.5 Docker Compose Overview . . . 12
4.6 Conclusion . . . 12

5 Caching Mechanism **12**
5.1 Concept of Caching . . . 13
5.2 Redis as a Caching Solution . . . 13
5.3 Caching in This Project . . . 13
5.4 Future Caching Strategy . . . 13
5.5 Conclusion . . . 14

6 Testing and Validation **14**
6.1 Testing Approach . . . 14
6.2 Backend API Testing . . . 15
6.3 Frontend Testing . . . 15
6.4 End-to-End Testing . . . 16
6.5 Validation Results . . . 17
6.6 Conclusion . . . 17

7 Performance and Limitations **17**
7.1 Performance Evaluation . . . 17
7.2 Limitations of the Current System . . . 18
7.3 Scalability Considerations . . . 18
7.4 Conclusion . . . 18

8 Security Considerations **18**
8.1 Authentication Mechanism . . . 18
8.2 Password Security . . . 19
8.3 Role-Based Access Control . . . 19
8.4 Secure Communication . . . 19

8.5 Additional Security Considerations 19
8.6 Future Security Enhancements 19
8.7 Conclusion 20

9 Future Improvements 20
9.1 Containerization Using Docker 20
9.2 Caching with Redis 21
9.3 Performance Monitoring 21
9.4 Enhanced Security Features 21
9.5 Real-Time Features 21
9.6 Mobile Application Integration 21
9.7 Scalability Enhancements 21
9.8 Conclusion 21

10 Conclusion 21

11 Live System Links 22

1 Introduction

This document presents Deliverable 5 of the Healthcare Web Application project. This phase focuses on the deployment, testing, and final evaluation of the complete system.

In previous deliverables, the system requirements, design, backend implementation, and frontend integration were successfully completed. This deliverable demonstrates the real-world deployment of the application using cloud platforms.

The application has been deployed using Vercel for the frontend and Render for the backend, with MongoDB Atlas as the database. This ensures accessibility, scalability, and real-world usability.

2 System Overview

The Healthcare Web Application is a complete full-stack web-based system designed to support three major user roles: patients, doctors, and administrators. The system was developed incrementally across the previous deliverables, where the project requirements were first established, followed by architectural design, backend implementation, and frontend integration. In this final phase, those completed modules are presented as a unified, deployed, and operational software system.

The overall objective of the system is to provide a secure, scalable, and user-friendly healthcare platform through which patients can register, log in, book appointments, and view their records; doctors can manage appointments and patient interactions; and administrators can oversee system-level operations. This real-world deployment confirms that the project has moved beyond design and development into a functional production-ready environment.

The implemented system follows a three-tier architecture, which separates presentation, application logic, and data management into independent layers. This architectural approach improves maintainability, modularity, and future scalability.

- **Frontend Layer:** Developed using React.js with Vite, this layer provides the user interface for authentication, dashboards, and appointment-related interactions.
- **Backend Layer:** Implemented using Node.js and Express.js, this layer handles RESTful API processing, business logic, authentication, authorization, and communication with the database.
- **Data Layer:** MongoDB Atlas is used as the cloud-hosted database to store user information, appointment records, and other application data in a scalable document-oriented format.

The communication flow of the deployed system is straightforward and efficient. Users interact with the frontend through a web browser, the frontend sends HTTP requests to the backend APIs, and the backend processes those requests before storing or retrieving data from MongoDB Atlas. Responses are then returned to the frontend and displayed dynamically to the user.

This deliverable focuses on the deployment and validation of this integrated architecture in a real cloud environment. The frontend has been deployed on Vercel, the backend has been deployed on Render, and the database is hosted on MongoDB Atlas. As a result, the system now demonstrates complete end-to-end functionality in a live setting.

Figure 1 presents the overall architecture of the Healthcare Web Application. For consistency with the earlier design document, the same architectural view is reused here to illustrate how the implemented and deployed components remain aligned with the original system design.

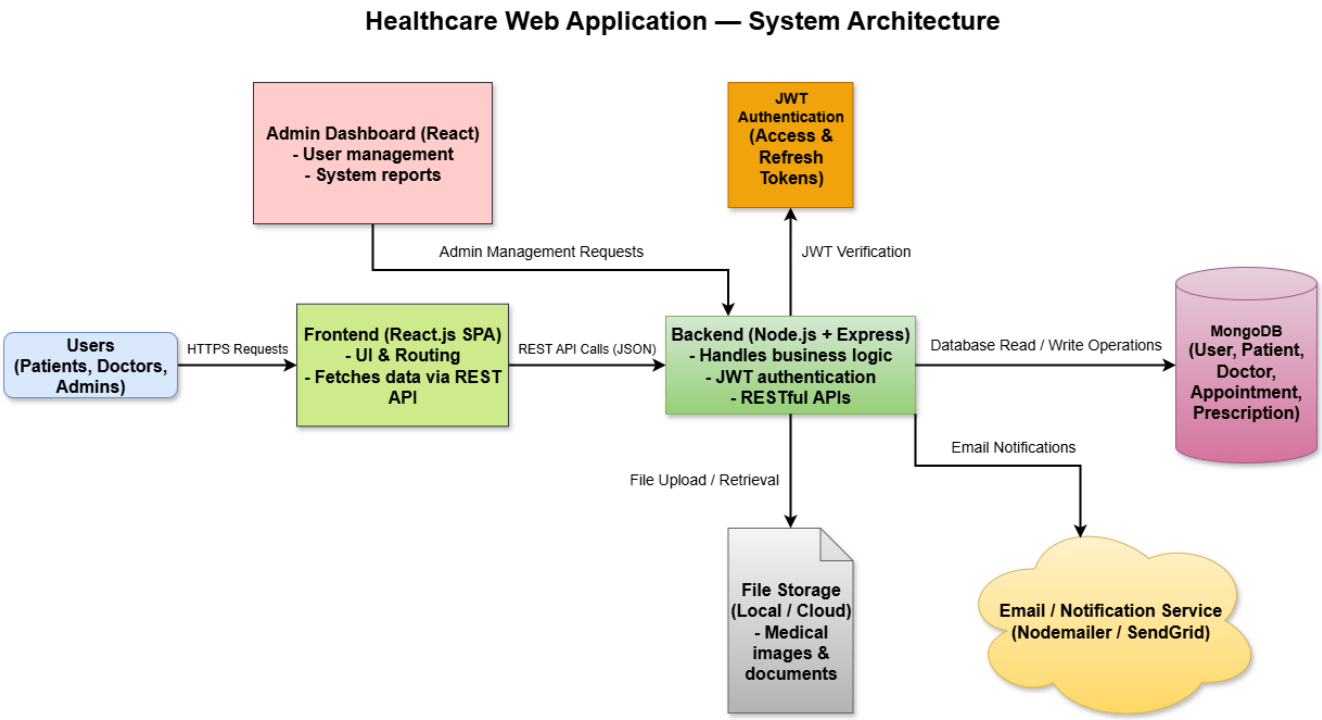

Figure 1: Overall system architecture of the Healthcare Web Application showing the interaction among the React frontend, Node.js/Express backend, and MongoDB database

3 Deployment Architecture

This section describes how the Healthcare Web Application has been deployed into a real-world cloud environment. The deployment transforms the previously developed system into a live, accessible, and production-ready application.

The system follows a distributed cloud deployment model, where each component is hosted independently using modern cloud platforms. This approach ensures scalability, reliability, and separation of concerns.

3.1 Deployment Overview

The deployed system consists of three major components:

- **Frontend Deployment:** The user interface is deployed on Vercel, a cloud platform optimized for frontend applications. It provides fast global content delivery and seamless integration with GitHub.
- **Backend Deployment:** The REST API server is deployed on Render as a web service. It handles business logic, authentication, and database communication.

- **Database Hosting:** MongoDB Atlas is used as a cloud-based database service to store application data securely and efficiently.

This separation allows each component to be scaled, updated, and maintained independently without affecting the others.

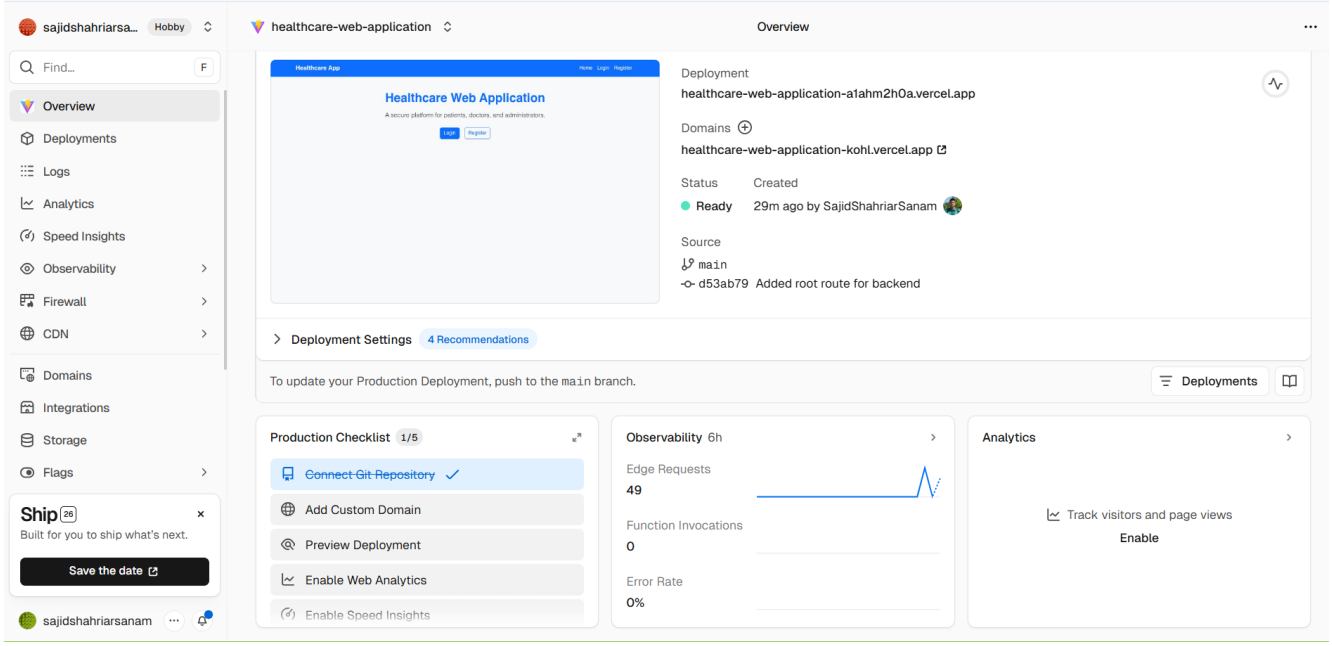

Figure 2: Frontend deployment overview on Vercel

Figure 2 shows the successful production deployment of the frontend on Vercel.

3.2 System Interaction Flow

The interaction flow of the deployed system is illustrated below:

1. The user accesses the application via the frontend URL hosted on Vercel.
2. The frontend sends API requests to the backend service hosted on Render.
3. The backend processes the request and interacts with MongoDB Atlas.
4. The database returns the requested data to the backend.
5. The backend sends the response back to the frontend.
6. The frontend dynamically updates the user interface.

This request-response cycle ensures real-time interaction and seamless user experience.

3.3 Deployment Platforms Used

Table 1: Deployment platforms used in the system

Component	Platform	Purpose
Frontend	Vercel	Hosting React application
Backend	Render	Hosting Node.js API server
Database	MongoDB Atlas	Cloud database storage

3.4 Environment Configuration

To ensure secure and flexible deployment, environment variables were used in the backend service. These variables store sensitive configuration data and prevent hardcoding of credentials.

The following environment variables were configured:

- **MONGO_URI:** Connection string for MongoDB Atlas
- **JWT_SECRET:** Secret key used for authentication and token generation
- **PORT:** Port on which the backend server runs

This approach improves security and allows easy updates without modifying source code.

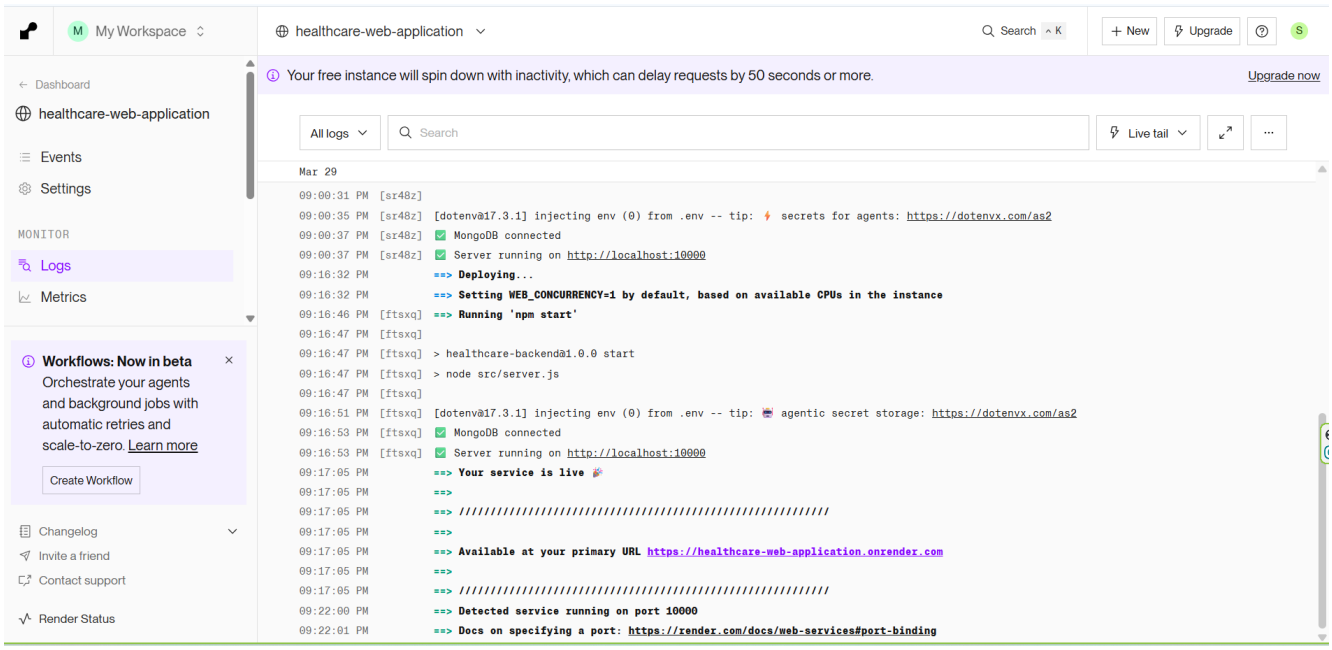

Figure 3: Backend deployment logs on Render showing successful startup and database connection

As shown in Figure 3, the backend server was successfully deployed and connected to MongoDB Atlas.

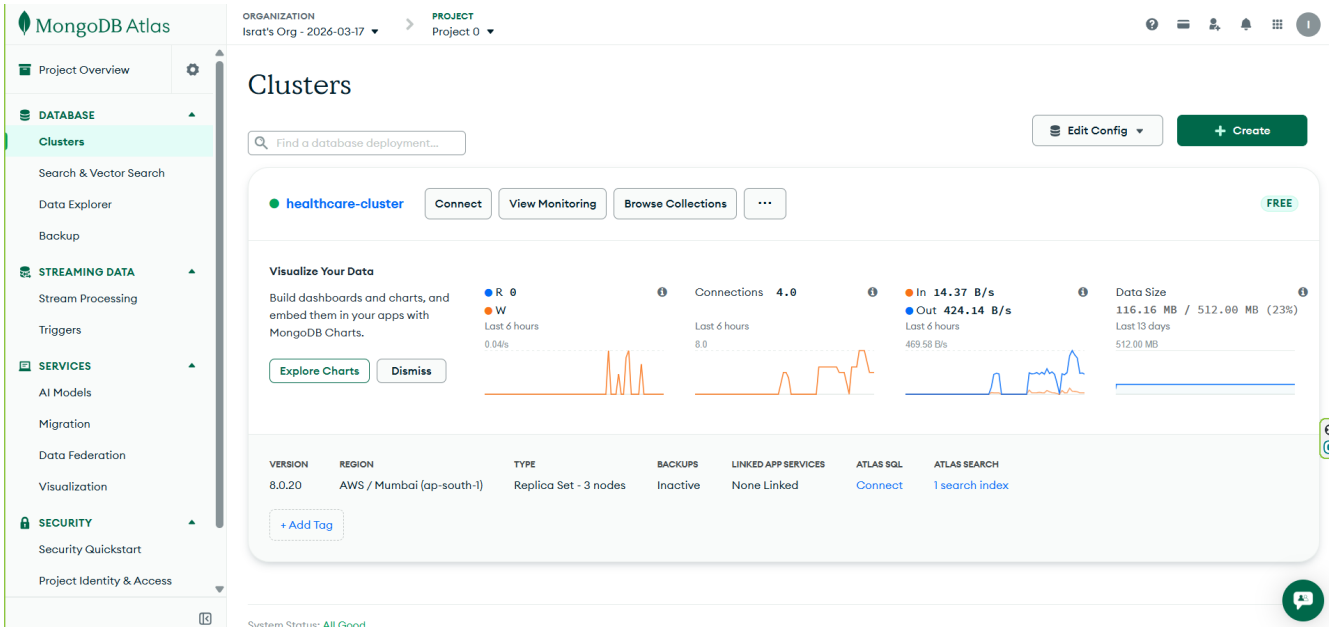

Figure 4: MongoDB Atlas cluster used as the cloud database for the deployed system

3.5 Live Deployment Links

The system is fully deployed and accessible online. The following links provide access to the application:

- **Frontend URL (Vercel):** `https://healthcare-web-application-kohl.vercel.app/`
- **Backend URL (Render):** `https://healthcare-web-application.onrender.com/`
- **GitHub Repository:** `https://github.com/SajidShahriarSanam/healthcare-web-applicat`

These links demonstrate that the system is not only implemented but also successfully deployed and operational.

3.6 Deployment Architecture Diagram

The deployment architecture of the system is illustrated in Figure 5. This diagram shows how the frontend, backend, and database communicate across cloud platforms.

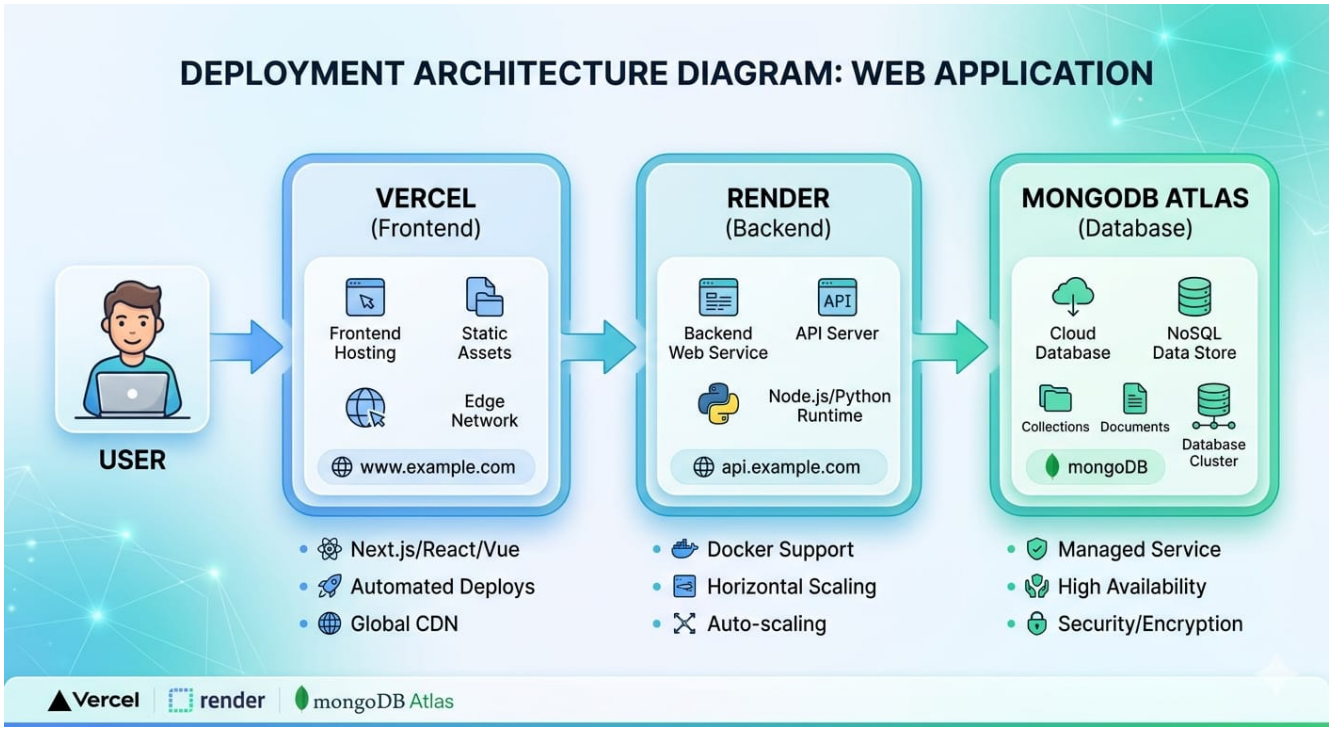

Figure 5: Deployment architecture showing Vercel (frontend), Render (backend), and MongoDB Atlas (database) interaction

4 Docker Containerization

Docker containerization is a modern approach used to package applications along with their dependencies into isolated environments known as containers. This ensures that the application runs consistently across different environments, including development, testing, and production.

In this project, containerization was considered as part of the deployment strategy to improve portability, scalability, and ease of deployment.

4.1 Concept of Containerization

Containerization allows the application to be bundled with all required libraries, runtime, and dependencies. Unlike traditional deployment, where environment configuration issues may arise, Docker ensures that the application behaves the same regardless of the host system.

Key advantages of containerization include:

- Consistent runtime environment
- Easy deployment across platforms
- Isolation of application components
- Simplified dependency management

4.2 Docker in This Project

Although the application was deployed using cloud platforms such as Vercel and Render, Docker support has been partially prepared in the project structure.

The frontend repository includes a Dockerfile, which allows the application to be containerized if needed. This ensures that the system can be easily migrated to container-based deployment environments in the future.

4.3 Sample Dockerfile (Frontend)

The following Dockerfile demonstrates how the frontend application can be containerized:

Listing 1: Sample Dockerfile for frontend deployment

```
# Use official Node.js image
FROM node:18

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy project files
COPY . .

# Build the application
RUN npm run build

# Expose port
EXPOSE 5173

# Start application
CMD ["npm", "run", "dev"]
```

4.4 Future Containerization Plan

In future enhancements, the complete system can be containerized using Docker Compose. This would include:

- Frontend container
- Backend container
- MongoDB container (optional for local setup)

This would allow the entire system to run using a single command and improve scalability for production-level deployments.

4.5 Docker Compose Overview

Docker Compose can be used to manage multiple containers simultaneously. A sample structure of the system would include:

- Frontend service (React)
- Backend service (Node.js)
- Database service (MongoDB)

This approach would significantly simplify deployment in real-world enterprise environments.

4.6 Conclusion

Although full container-based deployment was not implemented in this phase, the system has been designed with future containerization capability in mind. The existing structure ensures that Docker integration can be easily added without major modifications.

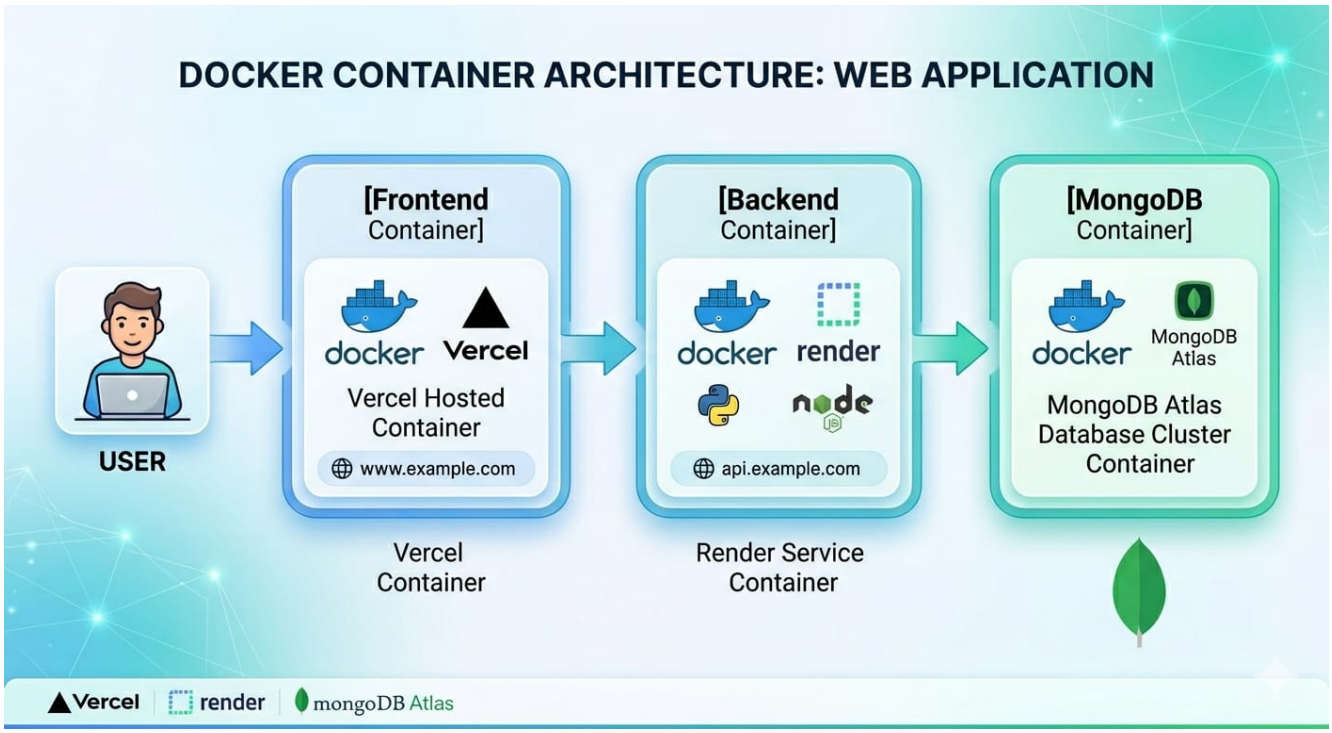

Figure 6: Conceptual containerized architecture using Docker

5 Caching Mechanism

Caching is an important technique used in modern web applications to improve performance, reduce response time, and minimize database load. It works by temporarily storing frequently accessed data so that repeated requests can be served faster without querying the database each time.

5.1 Concept of Caching

In a typical web application, every request may require database access, which can increase latency and reduce performance under heavy load. Caching addresses this issue by storing frequently requested data in memory.

Key benefits of caching include:

- Faster response time
- Reduced database load
- Improved scalability
- Better user experience

5.2 Redis as a Caching Solution

Redis (Remote Dictionary Server) is a popular in-memory data store commonly used for caching. It allows data to be stored and retrieved at high speed, making it suitable for real-time applications.

Redis can be used to cache:

- User session data
- Frequently accessed API responses
- Appointment data and user profiles

5.3 Caching in This Project

In the current implementation, caching has not been fully integrated due to time constraints. The system currently interacts directly with MongoDB Atlas for all data operations.

However, the architecture of the application supports the integration of a caching layer. By introducing Redis between the backend and the database, frequently requested data can be stored in memory, reducing response time and improving performance.

5.4 Future Caching Strategy

In future enhancements, Redis can be integrated into the backend to cache frequently accessed data. The proposed workflow is as follows:

1. The frontend sends a request to the backend.
2. The backend first checks Redis cache.
3. If data exists in cache, it is returned immediately.
4. If not, the backend queries MongoDB.
5. The result is stored in Redis for future use.
6. The response is sent to the frontend.

This strategy significantly reduces database queries and improves system efficiency.

5.5 Conclusion

Although caching has not been implemented in the current phase, it remains a valuable enhancement for improving system performance. The existing architecture allows easy integration of Redis in future deployments.

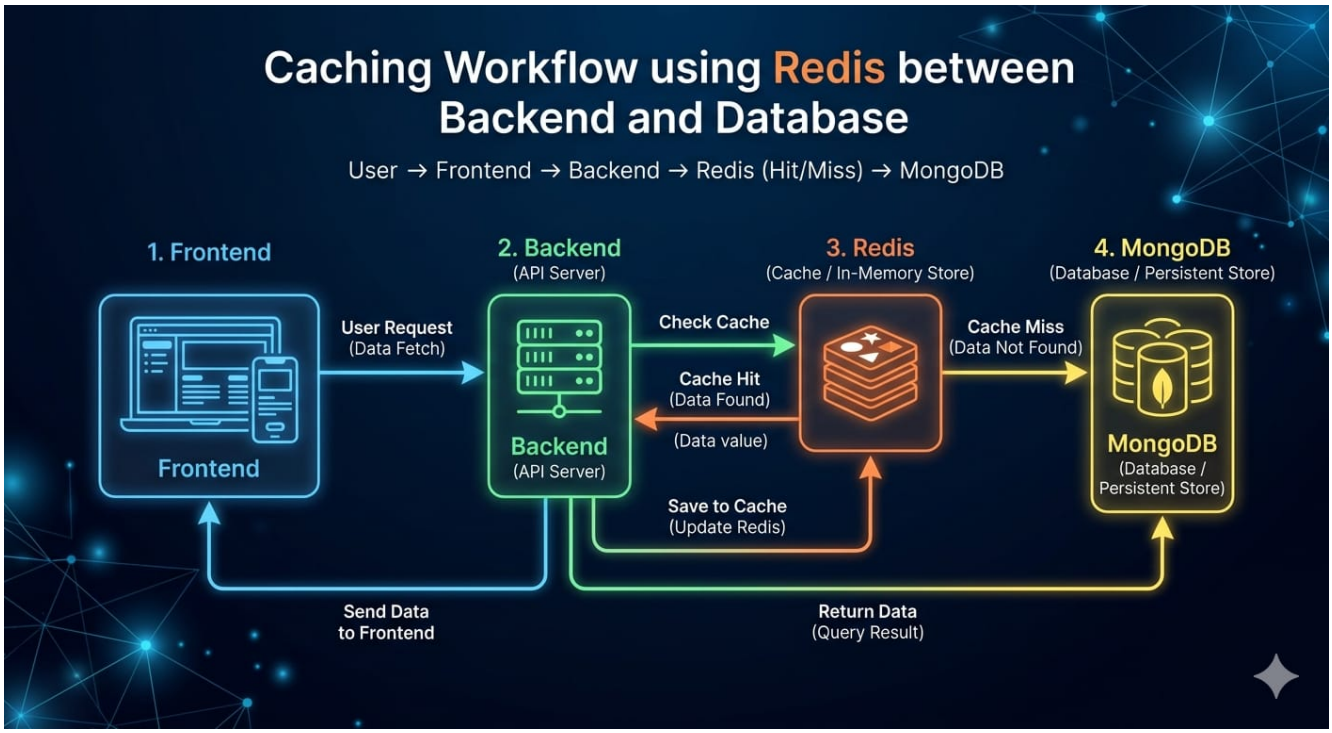

Figure 7: Caching workflow using Redis between backend and database

6 Testing and Validation

Testing and validation are essential steps to ensure that the Healthcare Web Application functions correctly in both development and deployed environments. In this phase, the system was tested extensively to verify its functionality, reliability, and performance.

6.1 Testing Approach

The system was tested using both backend API testing tools and frontend user interaction. This ensured that all components of the system work correctly both independently and as an integrated application.

The testing process included:

- Backend API testing using Postman
- Frontend testing using web browser
- End-to-end testing using the deployed application

6.2 Backend API Testing

All RESTful API endpoints were tested using Postman to verify correct request handling, authentication, and response generation.

The following functionalities were tested:

- User registration API
- User login API
- Appointment creation API
- Appointment retrieval API

All APIs returned appropriate responses, including success messages, authentication tokens, and error handling where necessary.

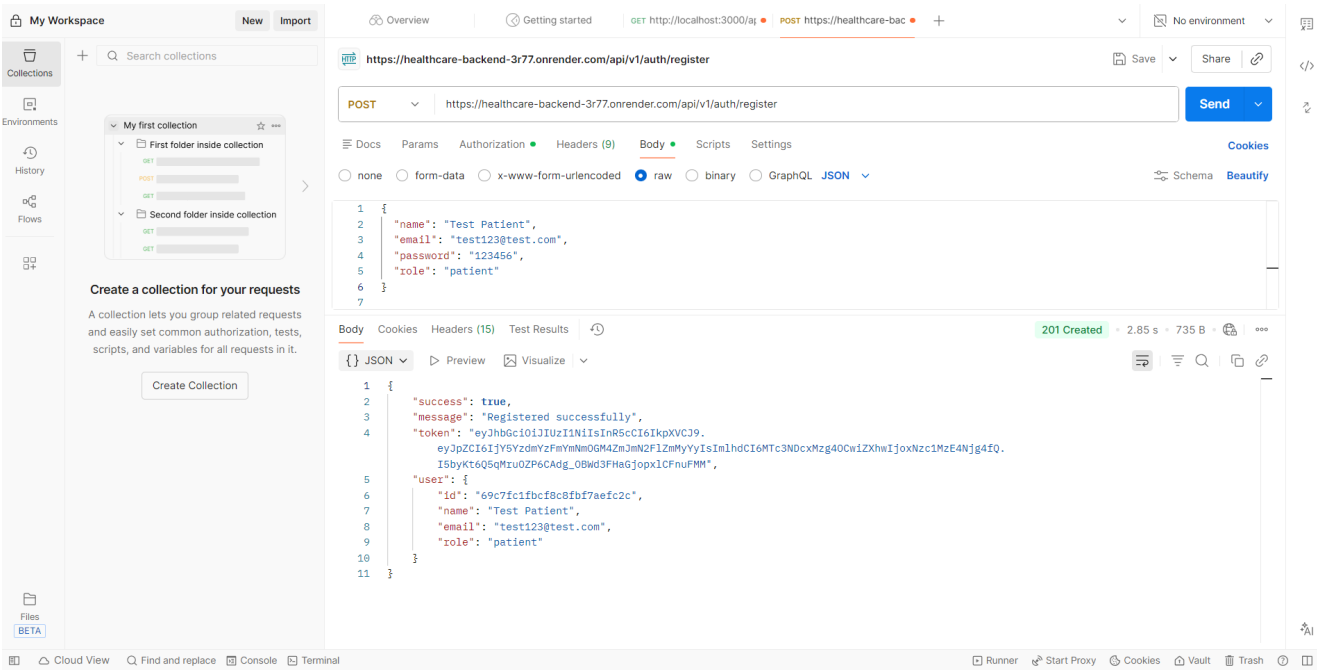

Figure 8: Testing API endpoints using Postman

6.3 Frontend Testing

The frontend application was tested through browser-based interaction to verify user interface functionality and API integration.

The following features were tested:

- User registration and login interface
- Role-based dashboard navigation
- Appointment booking functionality
- Viewing appointment history

The user interface responded correctly, and data was successfully retrieved from the backend.

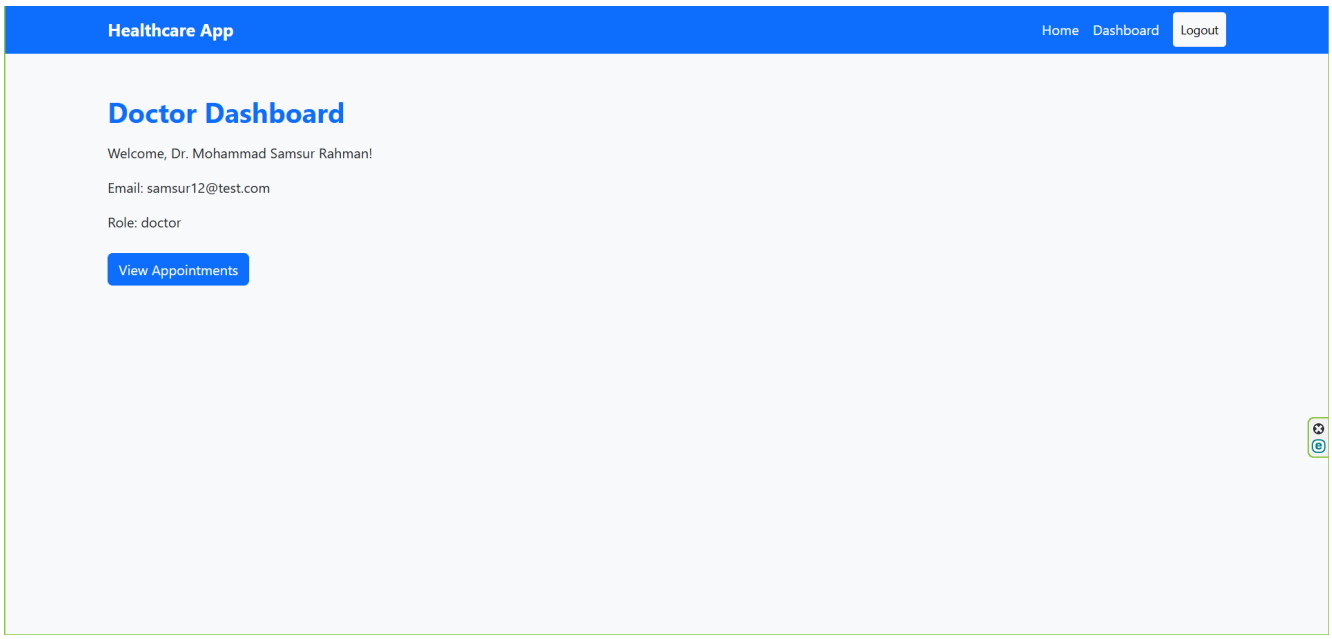

Figure 9: Frontend testing showing user dashboard and appointment functionality

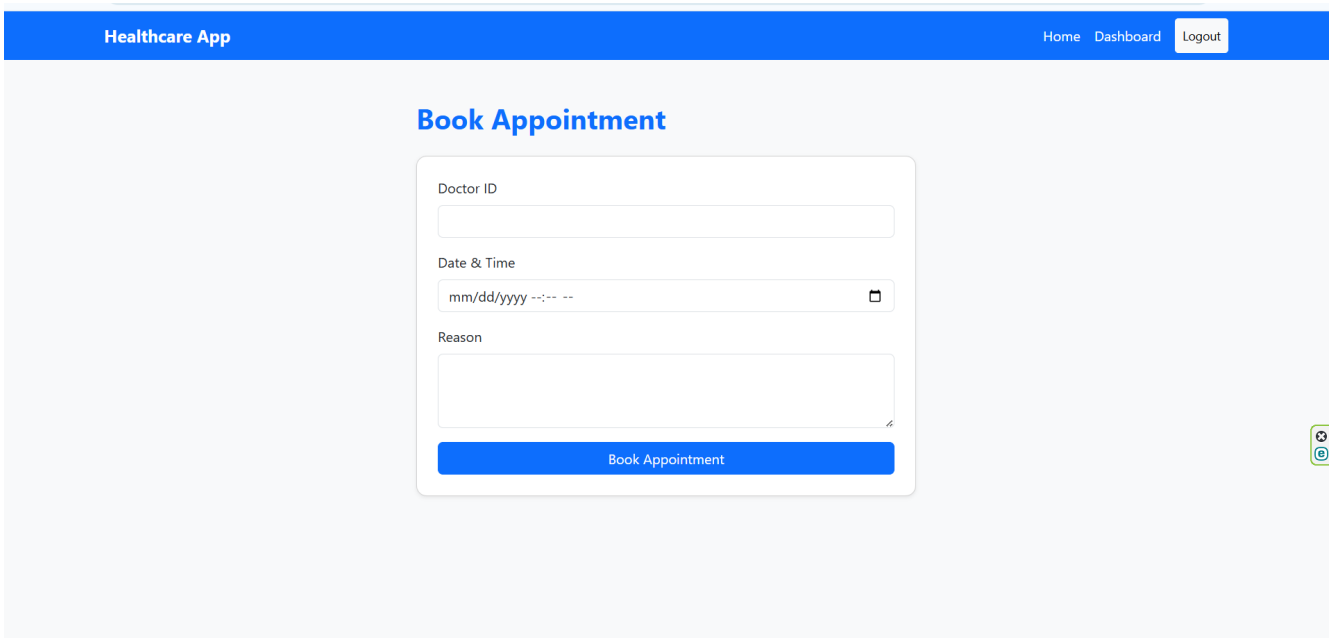

Figure 10: Frontend testing showing user dashboard and appointment functionality

6.4 End-to-End Testing

End-to-end testing was performed using the live deployed application to ensure that all system components work together correctly.

The following workflow was successfully tested:

1. User registers and logs into the system

2. User accesses the dashboard
3. User books an appointment
4. Appointment data is stored in the database
5. User views appointment history

This confirms that the deployed system is fully functional in a real-world environment.

6.5 Validation Results

The results of the testing phase indicate that:

- All core features are functioning correctly
- Backend and frontend integration is successful
- Data is accurately stored and retrieved from the database
- The deployed system performs reliably under normal usage

6.6 Conclusion

The testing and validation process confirms that the Healthcare Web Application meets its functional requirements and operates successfully in a live deployment environment.

7 Performance and Limitations

The performance of the Healthcare Web Application was evaluated based on responsiveness, system behavior under normal usage, and interaction with cloud services.

7.1 Performance Evaluation

The deployed system demonstrates satisfactory performance under typical usage conditions. Key observations include:

- Fast loading of frontend pages through Vercel hosting
- Efficient API response time from the backend service
- Successful real-time interaction between frontend and backend
- Reliable data storage and retrieval from MongoDB Atlas

The use of modern technologies such as React, Node.js, and cloud-based infrastructure contributes to the overall responsiveness and usability of the system.

7.2 Limitations of the Current System

Despite successful deployment, the current implementation has several limitations due to resource constraints and the use of free-tier services.

- Backend hosted on Render may experience cold start delays due to free-tier sleep mode
- No caching mechanism is implemented, which may affect performance under heavy load
- The system is not yet optimized for high concurrency scenarios
- Limited monitoring and logging tools are used in the current deployment

7.3 Scalability Considerations

The system architecture supports future scalability. The separation of frontend, backend, and database allows independent scaling of each component.

With additional improvements such as load balancing, caching, and containerization, the system can be scaled to handle larger user bases and higher traffic.

7.4 Conclusion

Overall, the system performs reliably under normal usage conditions. While certain limitations exist, they are primarily due to deployment constraints and can be addressed in future enhancements.

8 Security Considerations

Security is a critical aspect of the Healthcare Web Application, as the system handles sensitive user information such as personal data and appointment records. To ensure data protection and secure communication, several security mechanisms have been implemented.

8.1 Authentication Mechanism

The system uses JSON Web Tokens (JWT) for user authentication. After successful login or registration, a token is generated and returned to the client. This token is used to access protected routes in the backend.

Each request to protected endpoints must include the following header:

- `Authorization: Bearer <JWT_TOKEN>`

This ensures that only authenticated users can access secure resources.

8.2 Password Security

User passwords are not stored in plain text. Instead, the system uses bcrypt to hash passwords before storing them in the database.

This approach ensures that even if the database is compromised, the original passwords cannot be easily retrieved.

8.3 Role-Based Access Control

The system implements role-based access control (RBAC) to restrict access to resources based on user roles.

The supported roles include:

- Patient
- Doctor
- Administrator

Each role has specific permissions. For example:

- Patients can create and view appointments
- Doctors can manage and update appointment status
- Administrators can access system-level functionalities

8.4 Secure Communication

All communication between the frontend and backend occurs over HTTPS, ensuring encrypted data transmission between the client and server.

8.5 Additional Security Considerations

The system also includes basic security practices such as:

- Input validation to prevent invalid data submission
- Error handling to avoid exposing sensitive system details
- Protected API routes using authentication middleware

8.6 Future Security Enhancements

Although the current implementation provides essential security features, additional improvements can be made in the future:

- Rate limiting to prevent brute-force attacks
- Helmet middleware for securing HTTP headers
- Two-factor authentication (2FA)
- Advanced logging and monitoring tools

8.7 Conclusion

The implemented security mechanisms ensure that the Healthcare Web Application maintains confidentiality, integrity, and controlled access to system resources. These measures provide a secure foundation for future system expansion.

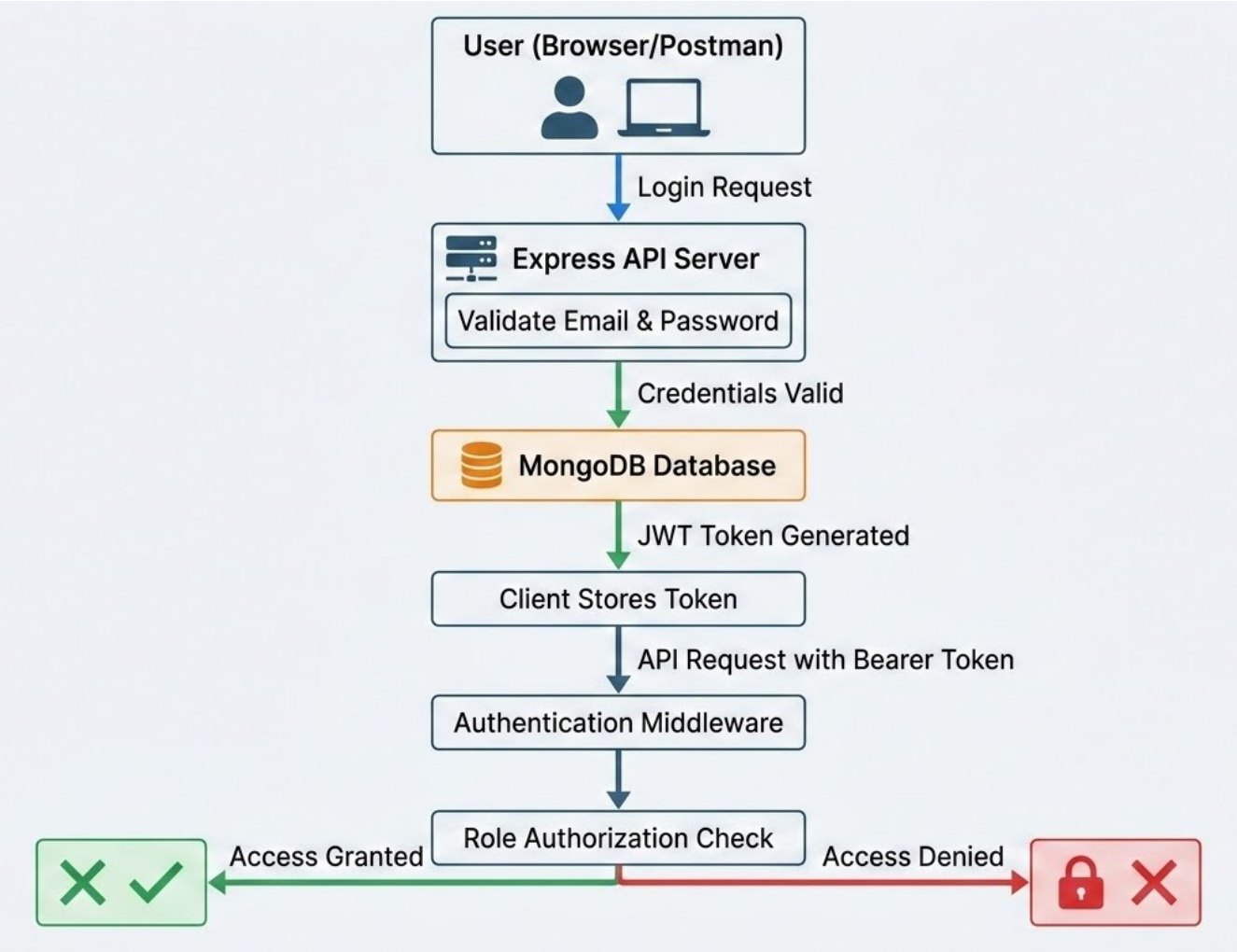

Figure 11: JWT-based authentication workflow between frontend and backend

9 Future Improvements

Although the Healthcare Web Application has been successfully developed and deployed, several enhancements can be implemented in future versions to improve performance, scalability, and functionality.

9.1 Containerization Using Docker

The system can be fully containerized using Docker to ensure consistent deployment across different environments. This would allow easier scaling and simplified configuration management.

9.2 Caching with Redis

A caching layer using Redis can be introduced to store frequently accessed data. This would reduce database load and significantly improve system performance, especially under high user traffic.

9.3 Performance Monitoring

Advanced monitoring tools such as Prometheus or cloud-based monitoring services can be integrated to track system performance, API response times, and server health.

9.4 Enhanced Security Features

Additional security measures can be implemented to further protect the system:

- Rate limiting to prevent brute-force attacks
- Two-factor authentication (2FA)
- Secure HTTP headers using Helmet middleware

9.5 Real-Time Features

Real-time functionalities such as appointment notifications, reminders, and chat between patients and doctors can be added using WebSocket or Firebase services.

9.6 Mobile Application Integration

A mobile version of the application can be developed using technologies such as React Native to improve accessibility for users.

9.7 Scalability Enhancements

The system can be scaled using load balancing, microservices architecture, and cloud auto-scaling to support a large number of users.

9.8 Conclusion

These improvements will transform the Healthcare Web Application into a more robust, scalable, and feature-rich system suitable for real-world healthcare environments.

10 Conclusion

The Healthcare Web Application has been successfully designed, developed, integrated, and deployed as a complete full-stack system. This project demonstrates the practical implementation of modern web technologies in building a real-world healthcare management platform.

Through the previous deliverables, the system evolved from requirement analysis and design to backend implementation and frontend integration. In this final phase, the

system has been deployed in a live cloud environment, making it accessible and functional for real users.

The application supports essential healthcare functionalities such as user authentication, role-based access control, appointment booking, and appointment management. The successful deployment using Vercel, Render, and MongoDB Atlas confirms that the system operates effectively in a production-like environment.

Overall, the project achieves its objectives by delivering a scalable, secure, and user-friendly healthcare web application. The system provides a strong foundation for further enhancements and real-world adoption.

11 Live System Links

The deployed system is accessible through the following links:

- **Frontend (Vercel):** `https://healthcare-web-application-kohl.vercel.app`
- **Backend (Render):** `https://healthcare-web-application.onrender.com`
- **GitHub Repository:** `https://github.com/SajidShahriarSanam/healthcare-web-applicat`

These links demonstrate that the application is fully deployed and operational in a real-world environment.